ABIDA

Autonomous Ops Intelligence Decision Agent

PROJECT TECHNICAL REPORT

Project Title:	ABIDA – Autonomous Ops Intelligence Decision Agent
Report Type:	Technical Architecture & Implementation Report
Name:	Syeda Noor Fatima
Reg No.	202269

1. Executive Summary

ABIDA (Autonomous Ops Intelligence Decision Agent) is a production-grade autonomous AI system engineered to transform raw operational business data into actionable intelligence. The system combines state-of-the-art large language model reasoning with a curated tool ecosystem, robust security guardrails, and a persistent long-term memory architecture.

Built on a LangGraph state machine backbone and exposed via a FastAPI web service, ABIDA is designed to operate autonomously while maintaining Human-in-the-Loop (HITL) checkpoints for high-stakes decisions. The project encompasses the full engineering lifecycle — from AI agent design through containerized deployment and automated quality assurance via CI/CD pipelines.

Orchestration	LLM Backend	Vector Store	API Layer	Deployment
LangGraph	**Llama 3.3 70B**	**ChromaDB**	**FastAPI**	**Docker**

2. Project Objectives

The core objectives driving the design and implementation of ABIDA are:

- Autonomous Operational Intelligence: Enable the system to independently analyze business KPI data, identify anomalies, and surface actionable insights without requiring manual intervention at every step.
- Security-First Architecture: Implement a multi-layered security posture that intercepts and neutralizes malicious inputs, including prompt injection attacks, before they reach the reasoning core.
- SOP-Grounded Decision Making: Anchor all agent recommendations in verifiable company Standard Operating Procedures through Retrieval-Augmented Generation (RAG), eliminating ungrounded hallucinations.
- Persistent Organizational Memory: Build a long-term memory system allowing the agent to retain, retrieve, and compare analytical findings across sessions and time periods.
- Production-Grade Quality: Package the system for real-world deployment with automated evaluation pipelines that block quality regressions before they reach production.

3. Core Architecture & Frameworks

ABIDA is structured as a multi-layer architecture where each layer has a distinct responsibility, ensuring separation of concerns and enabling independent scaling of each component.

3.1 LangGraph State Machine

The agent's reasoning and execution flow is governed by a structured LangGraph state machine. This design choice provides deterministic, auditable control flow — every step the agent takes is a named, observable node in the graph.

Component	Description
guardrail	The entry node. Acts as a security firewall, analyzing every incoming request for prompt injection patterns, jailbreak attempts, or malicious payloads. Requests that fail this check are blocked before any LLM call is made.
agent	The core reasoning engine. Receives sanitized inputs and leverages the LLM (Llama 3.3-70B via Groq) to plan actions, interpret tool results, and formulate final responses.
tools	The execution node. Dispatches calls to the agent's registered tools based on the reasoning engine's decisions and returns structured results back into the graph state.

3.2 API Layer (FastAPI)

The agent is exposed as a production web service through two complementary FastAPI endpoints, designed to serve different client interaction patterns:

- **/chat – Synchronous REST Endpoint**
  - Accepts a user query, runs the complete LangGraph pipeline, and returns the final synthesized answer as a standard JSON response. Ideal for programmatic integrations and batch queries.
- **/stream – Server-Sent Events (SSE) Streaming Endpoint**
  - Streams the agent's execution in real-time, emitting events for each graph node as it executes (e.g., 'guardrail_passed', 'tool_called', 'agent_reasoning'). Delivers the final answer as the terminal event. Ideal for interactive UI dashboards requiring live feedback.

3.3 LLM Provider

ABIDA uses ChatGroq as its LLM provider, specifically the llama-3.3-70b-versatile model. This selection balances high reasoning capability with cost efficiency and inference speed, making it appropriate for production workloads. The architecture is provider-agnostic by design, supporting local Llama instances as a fallback.

4. Tool Ecosystem & Capabilities

The agent is equipped with four purpose-built tools that collectively cover the full spectrum of operational analysis tasks — from raw data ingestion to policy retrieval to long-term knowledge management.

4.1 KPI Analysis Tool (analyze_kpis)

The primary data analysis tool, designed to ingest and process structured business data from CSV sources (e.g., Global Superstore datasets). Key capabilities include:

- Computation of average shipping delays across all orders and regional segments
- Automatic flagging of product lines and regions with profit margins falling below the 5% threshold

- Identification of problematic geographic regions based on composite performance metrics
- Powered by pandas and numpy, ensuring robust handling of large datasets

Technical Implementation Note

The KPI analyzer is designed as a stateless function that accepts a file path or data source reference as input and returns a structured JSON summary of findings. This ensures the tool is independently testable and can be swapped for different data sources without modifying the agent graph.

4.2 RAG Grounding Tool (grounding_search)

Implements Retrieval-Augmented Generation to anchor the agent's recommendations in verified company policy documents and SOPs. Architecture:

- Vector Store: ChromaDB is used to store embedded representations of company PDFs, SOPs, and policy documents
- Embedding Model: sentence-transformers generates high-quality semantic embeddings for both documents at index time and queries at retrieval time
- Workflow: When the agent identifies an operational issue, it automatically queries the knowledge base for relevant remediation policies before formulating a recommendation

4.3 Long-Term Memory System

ABIDA implements a two-tier memory architecture that persists knowledge across sessions:

Component	Description
Short-Term Memory	Implemented via LangGraph's session checkpointer. Maintains conversational context and tool call history within a single active session thread.
store_analysis_result	Enables the agent to autonomously generate and persist executive summaries of its analytical findings into a dedicated ChromaDB history collection after completing an analysis.
retrieve_past_analyses	Allows the agent to query its historical memory store, enabling temporal comparisons such as 'Has our average shipping delay improved since last quarter?' across multiple sessions.

4.4 Human-in-the-Loop (HITL) Checkpoints

To prevent autonomous agents from taking irreversible high-risk actions without oversight, the LangGraph graph is configured with interrupt points that pause execution at critical junctures:

- The graph pauses before any invocation of store_analysis_result, preventing unsupervised writes to long-term memory
- A human operator is presented with the pending action and the data to be written
- The operator can approve, reject, or modify the data before execution resumes
- All HITL decisions are logged, creating a complete audit trail of human oversight actions

5. Industrial Packaging & Deployment

5.1 Containerization

The project is fully containerized using a multi-stage Docker build based on the python:3.11-slim base image. The Dockerfile is optimized for layer caching — dependency installation layers are separated from application code layers, ensuring that code changes do not invalidate the dependency cache and rebuild times remain minimal.

5.2 Service Orchestration (Docker Compose)

A docker-compose.yml file orchestrates the full application stack as two distinct, independently manageable services:

Component	Description
FastAPI Agent Service	The main application container hosting the LangGraph agent and FastAPI web server. Depends on the ChromaDB service for vector operations.
ChromaDB Service	A standalone, dedicated ChromaDB server container providing persistent vector storage for both the RAG knowledge base and the long-term memory collections.

5.3 Persistence & Security

Two critical production requirements are addressed at the infrastructure level:

- Data Persistence: Docker Volumes are mounted to the ChromaDB service, ensuring that all embedded knowledge, RAG documents, and long-term memory survive container restarts and redeployments.
- Secrets Management: Sensitive credentials, including the GROQ_API_KEY, are never baked into the container image. They are injected purely at runtime via environment variables, keeping the image artifact secret-free and safe for distribution.

6. Automated Quality Gates & CI/CD Pipeline

A core principle of ABIDA's engineering philosophy is that agent quality degradation should be caught automatically, before it reaches production. A full automated evaluation pipeline achieves this.

6.1 Evaluation Framework (DeepEval)

The DeepEval framework is used to measure two critical dimensions of agent quality:

Component	Description
Faithfulness	Measures whether the agent's responses are grounded in the retrieved context and source documents, detecting hallucinations

	where the agent fabricates information not present in its tools or knowledge base.
Answer Relevancy	Measures whether the agent's final response actually addresses the user's original query, catching cases where the agent provides technically correct but contextually irrelevant answers.

6.2 Headless CI Evaluation Script (run_eval.py)

A dedicated evaluation script is designed for automated, headless execution in CI environments:

- Reads all API keys and configuration from environment variables — zero hardcoded secrets
- Executes the agent against a curated test suite defined in eval_tests.json, containing ground-truth question-answer pairs covering all major agent capabilities
- Compares measured metrics against versioned thresholds defined in eval_thresholds.json (e.g., minimum Faithfulness score of 0.7)
- Exits with code 1 if any metric falls below its threshold, enabling the CI system to fail the build

6.3 GitHub Actions Pipeline

A .github/workflows/main.yml pipeline provides continuous automated quality enforcement:

- Trigger: Automatically executes on every push to the main branch and on all pull requests targeting main
- Process: Spins up the evaluation environment, injects required secrets from GitHub Secrets, and runs run_eval.py against the latest code
- Gate: If the agent's measured Faithfulness or Answer Relevancy scores drop below the versioned thresholds, the pipeline fails with exit code 1, blocking the merge or deployment
- Outcome: Developers receive immediate feedback on quality regressions, and no degraded code can reach production undetected

7. Technology Stack Summary

Layer	Technology	Purpose
Agent Orchestration	LangGraph	State machine workflow management
LLM Reasoning	Llama 3.3-70B (Groq)	Core intelligence and reasoning engine
Web Framework	FastAPI	REST and SSE API endpoints
Vector Database	ChromaDB	RAG knowledge base & long-term memory
Embeddings	sentence-transformers	Semantic document and query embedding
Data Analysis	pandas + numpy	KPI computation and data processing

Containerization	Docker + Compose	Deployment and service orchestration
Evaluation	DeepEval	Faithfulness and relevancy measurement
CI/CD	GitHub Actions	Automated quality gates on every push

8. Key Design Decisions & Rationale

8.1 Guardrail as a First-Class Graph Node

Security was implemented as a dedicated, observable LangGraph node rather than as middleware or a pre-processing wrapper. This design ensures that security checks are versioned alongside agent logic, appear in execution traces for auditability, and are testable in isolation — making them a first-class concern rather than an afterthought.

8.2 Two-Tier Memory Architecture

The split between short-term session checkpointing and long-term ChromaDB memory reflects different access patterns and lifetimes. Session memory needs to be fast, ephemeral, and scoped to a conversation. Long-term memory needs to be persistent, queryable by semantic similarity, and accessible across sessions. Using distinct mechanisms for each avoids the performance and complexity penalties of forcing one system to handle both requirements.

8.3 HITL Gate on Memory Writes

Allowing an autonomous agent to write permanently to its own memory store without human review introduces a class of failure where compounding errors accumulate in memory and progressively degrade future reasoning. Placing a mandatory HITL interrupt before any persist operation ensures a human validates the accuracy and appropriateness of what the agent commits to long-term knowledge.

8.4 Versioned Evaluation Thresholds

Evaluation thresholds are stored in a committed eval_thresholds.json file rather than hardcoded in the CI pipeline configuration. This means threshold changes are subject to code review, creating accountability for any relaxation of quality standards. It also allows thresholds to evolve with the project without modifying CI infrastructure.

9. Conclusion

ABIDA represents a complete, production-ready implementation of an autonomous operational intelligence agent. By combining the structured control flow of LangGraph with RAG grounding, persistent memory, security guardrails, and automated quality evaluation, the system addresses the full spectrum of requirements for deploying AI agents in real business environments.

The architecture prioritizes auditability, safety, and quality — not as post-hoc additions, but as structural properties of the design itself. The result is a system that can operate autonomously at scale while remaining explainable, correctable, and continuously validated against measurable quality standards.

Project Status Summary

The following components have been fully designed and implemented:

- LangGraph state machine with guardrail, agent, and tools nodes
- FastAPI service with /chat and /stream endpoints
- KPI analysis tool with pandas/numpy
- ChromaDB RAG knowledge base with sentence-transformer embeddings
- Two-tier memory system (session checkpointer + long-term ChromaDB)
- Human-in-the-Loop interrupt gate on memory writes
- Docker + Docker Compose production deployment
- DeepEval quality evaluation with versioned thresholds
- GitHub Actions CI/CD pipeline with automated quality gates